



南開大學
Nankai University

计算机学院
高级语言程序设计大作业

Nankai Rhythm

姓名：张宸笛号

学号：2412565

专业：计算机科学与技术

2026 年 5 月 14 日

目录

1 作业题目	3
2 开发环境与工具	3
2.1 开发环境	3
2.2 项目地址	3
3 课题要求与完成情况	3
3.1 课题要求	3
3.2 完成情况	4
4 系统总体设计	4
4.1 整体流程	4
4.2 场景设计	4
4.3 核心类设计	5
5 主要功能实现	5
5.1 四轨道节奏游戏机制	5
5.2 音符位置计算	5
5.3 音乐同步机制	6
5.4 谱面文件读取	6
5.5 判定算法	7
5.6 分数、准确率与评级	7
5.7 Combo 里程碑特殊效果	8
6 项目结构	9
7 关键代码说明	10
7.1 主循环	10
7.2 场景切换	10
7.3 谱面读取	10
8 运行效果	12
8.1 主菜单界面	12
8.2 游戏界面	13
8.3 10 Combo 特效	14
8.4 结算界面	15
9 测试与结果	15
9.1 功能测试	15
9.2 测试结果	16

10 开发过程中遇到的问题与解决方法	16
10.1 Esc 键直接关闭窗口的问题	16
10.2 音乐与谱面对齐问题	16
10.3 10 Combo 特效重复触发问题	17
10.4 谱面文件异常问题	17
11 版本迭代	17
12 AI 工具使用说明	17
13 收获与总结	17

1 作业题目

本项目题目为：

Nankai Rhythm：基于 C++ 的图形化节奏游戏

项目使用 C++ 语言和 raylib 图形库开发，实现了一个四轨道图形化节奏游戏。玩家需要根据屏幕中下落的音符，在音符到达判定线附近时按下对应按键。程序根据玩家按键与音符理论命中时间之间的误差，给出 Perfect、Good、Bad、Miss 等判定，并统计分数、连击数、准确率、最高分和最终评级。

本项目的主题选择结合了节奏游戏和电吉他音乐元素。相比传统的学生管理系统、图书管理系统等题目，本项目更强调实时交互、图形化表现和游戏反馈效果，具有较强的可演示性。

2 开发环境与工具

2.1 开发环境

本项目的主要开发环境如下：

表 1: 开发环境

项目	内容
操作系统	Windows 11
开发语言	C++17
编辑器	Visual Studio Code
编译器	MSVC
构建工具	CMake
图形库	raylib
版本管理	Git, GitHub

2.2 项目地址

项目代码托管地址如下：

- Gitee / GitHub 项目地址：<https://github.com/iodwad/Nankai-Rhythm/>
- B 站讲解视频地址：https://www.bilibili.com/video/BV18P596gEQS/?spm_id_from=333.1387.homepage.video_card.click

3 课题要求与完成情况

3.1 课题要求

本次大作业要求使用 C++ 语言完成一个图形化小程序，图形化平台不限，程序主题自选，并鼓励创新和创意。项目还需要提交代码仓库地址、讲解视频地址和项目报告。

3.2 完成情况

本项目完成了以下主要功能：

- 1. 实现图形化窗口、主菜单、帮助界面、游戏界面和结算界面；
- 2. 实现四轨道节奏游戏玩法，使用 D/F/J/K 四个按键分别对应四条轨道；
- 3. 实现音符从上向下移动，并在判定线附近接受玩家输入；
- 4. 实现 Perfect、Good、Bad、Miss 四类判定；
- 5. 实现分数、当前连击、最大连击、准确率和评级统计；
- 6. 实现 10 连击特殊视觉效果；
- 7. 实现音乐播放，并使用音乐播放时间驱动谱面同步；
- 8. 实现谱面文件读取，支持根据文本文件生成音符；
- 9. 实现最高分本地保存；
- 10. 使用面向对象方式组织程序结构。

4 系统总体设计

4.1 整体流程

程序整体流程如图 4.1 所示。程序启动后进入主菜单，玩家可以选择开始游戏、查看帮助或退出程序。进入游戏后，系统加载谱面和音乐，音符根据音乐播放时间下落。游戏结束后进入结算界面，显示得分、准确率和评级等信息。

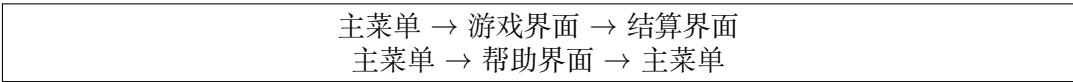


图 4.1: 程序整体流程

4.2 场景设计

本项目将不同界面抽象为不同场景。所有场景继承自抽象基类 `Scene`，并实现统一的 `update()`、`draw()` 和 `handleInput()` 接口。这样可以将不同界面的逻辑隔离，使主循环只需要调用当前场景的统一接口。

主要场景包括：

表 2: 场景类设计

类名	功能
<code>Scene</code>	抽象场景基类
<code>MenuScene</code>	主菜单界面
<code>HelpScene</code>	帮助说明界面
<code>PlayScene</code>	游戏主界面，负责核心玩法
<code>ResultScene</code>	结算界面，展示游戏结果

抽象基类 `Scene` 的核心设计如下：

```
1 class Scene {
2 public:
3     virtual void update(float dt) = 0;
4     virtual void draw() = 0;
5     virtual void handleInput() = 0;
6     virtual ~Scene() = default;
7 };
```

该设计体现了 C++ 中的继承与多态。主游戏类 `Game` 持有当前场景对象，并通过基类指针或智能指针调用派生类实现，从而实现不同界面之间的动态切换。

4.3 核心类设计

项目中的主要类如下：

表 3: 核心类及职责

类名	职责
<code>Game</code>	管理窗口、主循环和场景切换
<code>Scene</code>	场景抽象基类
<code>Note</code>	表示单个音符，包括轨道、命中时间和判定状态
<code>Chart</code>	负责读取谱面文件并管理音符列表
<code>ScoreManager</code>	管理分数、连击、准确率和评级
<code>PlayScene</code>	实现音符移动、玩家输入、判定和绘制
<code>ResultScene</code>	显示最终游戏结果

5 主要功能实现

5.1 四轨道节奏游戏机制

游戏界面包含四条竖直轨道，分别对应键盘上的 D、F、J、K 四个按键。谱面文件中的每一个音符记录了两个基本信息：

- `hitTime`：音符理论命中时间；
- `lane`：音符所在轨道，取值为 0、1、2、3。

玩家按键时，程序只在对应轨道中查找尚未判定的音符，并选择距离当前时间最近的音符进行判定，避免错误地判定其他轨道的音符。

5.2 音符位置计算

本项目没有简单地使用每帧累加位移来控制音符下落，而是根据当前谱面时间动态计算音符位置。设判定线纵坐标为 Y_j ，音符理论命中时间为 t_h ，当前谱面时间为 t_c ，音符速度为 v ，则音符纵坐标 y 的计算公式为：

$$y = Y_j - (t_h - t_c) \times v$$

当 $t_c = t_h$ 时, 有:

$$y = Y_j$$

此时音符刚好到达判定线。因此, 该算法可以保证音符位置与谱面时间直接绑定, 降低由于帧率波动带来的节奏误差。

5.3 音乐同步机制

项目后续版本中加入了音乐播放功能。游戏使用 raylib 的音乐流接口播放音乐, 并通过音乐播放时间作为统一时钟。设音乐当前播放时间为 t_m , 谱面偏移量为 $offset$, 则谱面时间 t_c 定义为:

$$t_c = t_m - offset$$

其中, $offset$ 用于处理音乐开头的准备时间或空白段。例如本项目使用的音乐约为 90 BPM, 长度约 25 秒。若音乐前方保留约 2 秒准备时间, 则可设置:

$$offset = 2.0$$

这样音乐播放到第 2 秒时, 谱面时间才从 0 开始。

音乐同步的核心逻辑可概括为:

```
1 UpdateMusicStream(music);
2 float musicTime = GetMusicTimePlayed(music);
3 float chartTime = musicTime - leadInTime;
```

之后, 音符下落、按键判定、自动 Miss 和游戏结束判断均使用 `chartTime`, 从而保证游戏节奏与音乐播放进度保持一致。

5.4 谱面文件读取

谱面文件采用简单文本格式, 每一行包含音符命中时间和轨道编号。例如:

```
1 # time lane
2 0.00 0
3 0.67 1
4 1.33 2
5 2.00 3
```

其中, 90 BPM 音乐的每拍间隔为:

$$\frac{60}{90} \approx 0.667 \text{ 秒}$$

因此, 在谱面设计时可以按照 0.67 秒作为基础拍间隔。为了提高程序鲁棒性, 谱面读取模块支持空行和注释行, 并会忽略非法数据。合法轨道编号只能为 0、1、2、3, 合法时间必须大于等于 0。读取完成后, 程序会按时间对音符排序。

5.5 判定算法

玩家按下按键后，程序计算当前谱面时间与音符理论命中时间之间的绝对误差：

$$\Delta t = |t_c - t_h|$$

根据误差大小进行判定：

表 4: 判定规则

判定	时间误差	分数	连击规则
Perfect	$\Delta t \leq 0.08s$	+1000	Combo + 1
Good	$\Delta t \leq 0.16s$	+600	Combo + 1
Bad	$\Delta t \leq 0.25s$	+200	Combo + 1
Miss	超过判定窗口	+0	Combo 清零

自动 Miss 的规则如下：

$$t_c - t_h > 0.25s$$

如果某个音符超过 Miss 窗口仍未被击中，则程序自动将其判定为 Miss，并将该音符标记为已判定，避免重复处理。

核心判定流程如下：

```

1 float delta = std::abs(currentTime - note.hitTime);
2
3 if (delta <= 0.08f) {
4     scoreManager.addPerfect();
5 } else if (delta <= 0.16f) {
6     scoreManager.addGood();
7 } else if (delta <= 0.25f) {
8     scoreManager.addBad();
9 }
10 note.judged = true;
```

5.6 分数、准确率与评级

分数由不同判定累加得到。准确率根据不同判定的权重计算：

$$accuracy = \frac{perfect \times 1.0 + good \times 0.7 + bad \times 0.3}{totalNotes}$$

最终评级规则如下：

表 5: 评级规则

准确率	评级
$\geq 95\%$	S
$\geq 90\%$	A
$\geq 80\%$	B
$\geq 70\%$	C
$< 70\%$	D

5.7 Combo 里程碑特殊效果

为了增强游戏反馈, 本项目加入了 Combo 里程碑特殊效果。当玩家连续命中达到 10、20、30 等 10 的倍数 Combo 时, 游戏界面中央会显示醒目的连击提示, 例如 10 COMBO!、20 COMBO!、30 COMBO!, 并配合轨道或判定线高亮, 持续约 1 秒。

实现该效果时需要避免简单使用如下逻辑:

```
1 if (combo >= 10 && combo % 10 == 0) {  
2     triggerEffect();  
3 }
```

因为游戏每一帧都会更新。如果玩家当前 Combo 停留在 10、20、30 等数值附近, 上述逻辑可能导致同一个里程碑特效被重复触发。为避免重复触发, 程序使用 `lastComboMilestoneEffect_` 记录上一次已经触发过特效的 Combo 数值。只有当当前 Combo 是 10 的倍数, 并且大于上一次触发记录时, 才会启动新的里程碑特效。

核心代码如下:

```
1 void PlayScene::applyJudgement(Judgement judgement) {  
2     score_.apply(judgement);  
3  
4     const int combo = score_.getCombo();  
5     if (combo >= 10 && combo % 10 == 0 &&  
6         combo > lastComboMilestoneEffect_) {  
7         lastComboMilestoneEffect_ = combo;  
8         comboMilestoneEffectCombo_ = combo;  
9         comboMilestoneEffectTimer_ = 1.0f;  
10    }  
11  
12    lastJudgement_ = judgement;  
13    judgementTextTimer_ = 0.45f;  
14 }
```

其中, `comboMilestoneEffectCombo_` 用于记录当前正在显示的里程碑 Combo 数值, 例如 10、20 或 30; `comboMilestoneEffectTimer_` 用于控制特效持续时间。每帧更新时, 程序会递减该计时器:

```
1 if (comboMilestoneEffectTimer_ > 0.0f) {  
2     comboMilestoneEffectTimer_ -= dt;  
3 }
```

绘制阶段根据 `comboMilestoneEffectTimer_` 判断是否显示特效文本,并根据 `comboMilestoneEffectCombo_` 拼接显示内容。这样可以在多个连击里程碑处产生反馈,同时避免同一个 Combo 数值重复触发特效。

游戏重新开始时,需要重置相关状态:

```
1 lastComboMilestoneEffect_ = 0;
2 comboMilestoneEffectCombo_ = 0;
3 comboMilestoneEffectTimer_ = 0.0f;
```

通过这种方式,玩家每达到一个新的 10 连击倍数,都会获得一次明确的视觉反馈,提升了游戏中的节奏感和成就感。

6 项目结构

项目目录结构如下:

```
1 NankaiRhythm/
2 |-- CMakeLists.txt
3 |-- README.md
4 |-- assets/
5 |   |-- charts/
6 |   |   |-- easy.txt
7 |   |-- music/
8 |   |   |-- demo.ogg
9 |   |-- save/
10 |       |-- highscore.txt
11 |-- src/
12 |   |-- main.cpp
13 |   |-- Game.h
14 |   |-- Game.cpp
15 |   |-- Scene.h
16 |   |-- MenuScene.h
17 |   |-- MenuScene.cpp
18 |   |-- HelpScene.h
19 |   |-- HelpScene.cpp
20 |   |-- PlayScene.h
21 |   |-- PlayScene.cpp
22 |   |-- ResultScene.h
23 |   |-- ResultScene.cpp
24 |   |-- Note.h
25 |   |-- Note.cpp
26 |   |-- Chart.h
27 |   |-- Chart.cpp
28 |   |-- ScoreManager.h
29 |   |-- ScoreManager.cpp
```

其中, `assets/charts/easy.txt` 存放谱面文件, `assets/music/demo.ogg` 存放音乐文件, `assets/save/highscore.txt` 用于保存最高分。

7 关键代码说明

7.1 主循环

主循环由 `Game` 类负责。程序每一帧依次处理输入、更新当前场景并绘制画面。伪代码如下：

```
1 while (!WindowShouldClose() && !shouldQuit_) {
2     float dt = GetFrameTime();
3
4     currentScene->handleInput();
5     currentScene->update(dt);
6
7     BeginDrawing();
8     ClearBackground(BLACK);
9     currentScene->draw();
10    EndDrawing();
11 }
```

为了避免 raylib 默认将 `Esc` 键作为关闭窗口键，程序在初始化窗口后禁用了默认退出键，使 `Esc` 的含义完全由不同场景自行处理。

```
1 InitWindow(screenWidth, screenHeight, "Nankai Rhythm");
2 SetExitKey(0);
```

7.2 场景切换

主菜单、帮助界面、游戏界面和结算界面均由 `Game` 类统一切换。例如：

```
1 void Game::goToMenu() {
2     currentScene = std::make_unique<MenuScene>(*this);
3 }
4
5 void Game::startGame() {
6     currentScene = std::make_unique<PlayScene>(*this);
7 }
8
9 void Game::showResult(const ScoreManager& score) {
10    currentScene = std::make_unique<ResultScene>(*this, score);
11 }
```

使用 `std::unique_ptr` 可以自动管理场景对象生命周期，避免手动 `new` 和 `delete` 带来的内存管理问题。

7.3 谱面读取

谱面读取时，程序逐行读取文件内容，跳过空行和注释行，并对每一行解析时间和轨道编号。若解析失败或数据不合法，则忽略该行。读取后按时间排序。

```
1 bool Chart::loadFromFile(const std::string& filename) {
```

```
2     std::ifstream fin(filename);
3     if (!fin.is_open()) {
4         loadFallbackChart();
5         return false;
6     }
7
8     std::string line;
9     while (std::getline(fin, line)) {
10         if (line.empty() || line[0] == '#') continue;
11
12         std::istringstream iss(line);
13         float time;
14         int lane;
15         if (!(iss >> time >> lane)) continue;
16         if (time < 0.0f || lane < 0 || lane > 3) continue;
17
18         notes.emplace_back(lane, time);
19     }
20
21     std::sort(notes.begin(), notes.end(),
22         [](const Note& a, const Note& b) {
23             return a.hitTime < b.hitTime;
24         });
25
26     if (notes.empty()) {
27         loadFallbackChart();
28         return false;
29     }
30     return true;
31 }
```

8 运行效果

8.1 主菜单界面



图 8.2: 主菜单界面

8.2 游戏界面

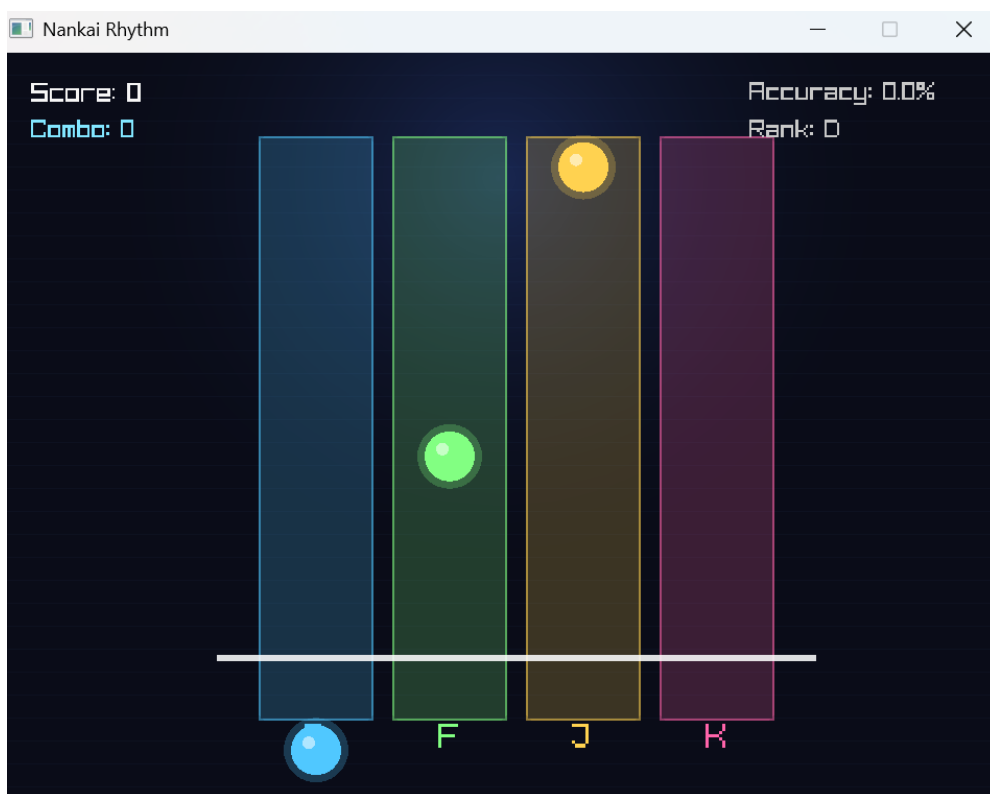


图 8.3: 游戏界面

8.3 10 Combo 特效

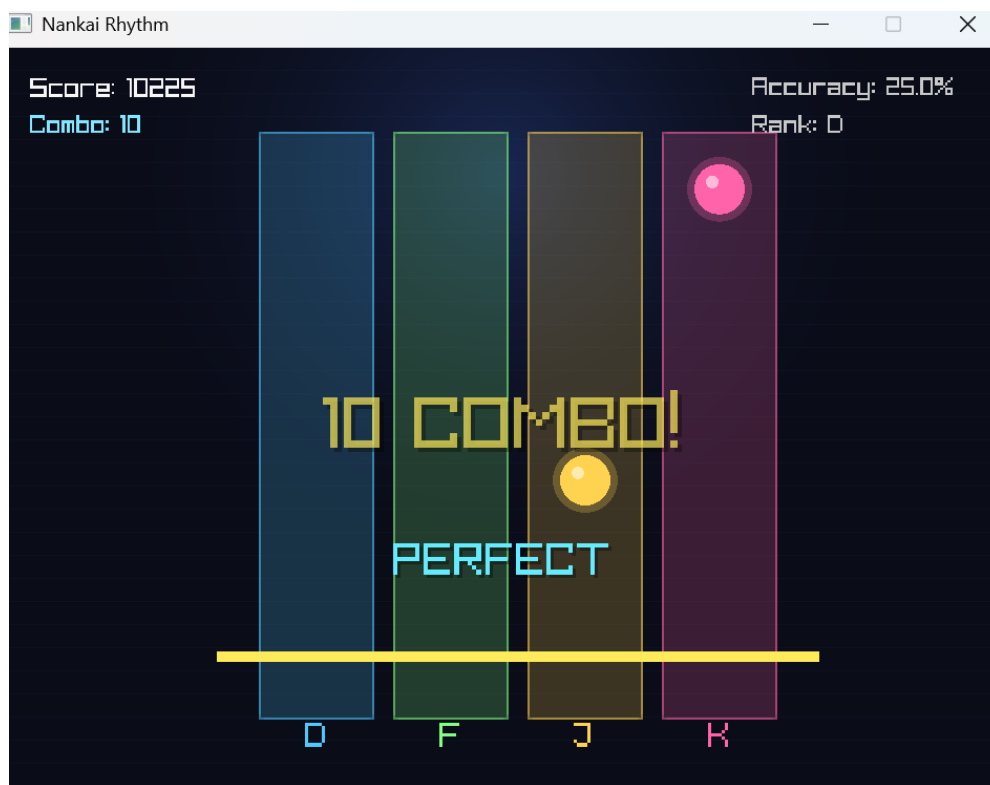


图 8.4: 10 Combo 特殊效果

8.4 结算界面



图 8.5: 结算界面

9 测试与结果

9.1 功能测试

本项目进行了如下功能测试：

表 6: 功能测试表

测试项目	测试内容	预期结果
主菜单测试	启动程序, 按 Enter	进入游戏界面
帮助界面测试	主菜单按 H, 帮助页按 Esc	进入帮助页并能返回主菜单
退出测试	主菜单按 Esc	程序退出
轨道输入测试	游戏中按 D/F/J/K	对应轨道产生按键反馈
判定测试	在音符接近判定线时按键	显示 Perfect/Good/Bad 判定
Miss 测试	故意不按键	音符超时后自动 Miss
Combo 测试	连续击中多个音符	Combo 连续增加
10 Combo 特效测试	连续击中 10 个音符	显示 10 Combo 特殊效果
结算测试	完成全部音符	显示分数、准确率和评级
重新开始测试	结算页按 Enter	重新开始游戏, 分数和状态重置
返回菜单测试	游戏中或结算页按 Esc	返回主菜单
最高分测试	获得高于历史记录的分	本地最高分文件更新
音乐同步测试	播放音乐并观察音符到达判定线时间	音符与音乐节奏基本一致

9.2 测试结果

经过测试, 程序能够正常完成从主菜单进入游戏、音乐播放、音符下落、按键判定、分数统计、结算展示和重新开始等完整流程。10 Combo 特效能够在玩家首次达到 10 连击时触发, 且不会在 10 连击之后每帧重复触发。最高分文件能够正常读取和写入。

10 开发过程中遇到的问题与解决方法

10.1 Esc 键直接关闭窗口的问题

在早期版本中, 游戏结束界面按下 Esc 键时, 程序会直接关闭, 而不是返回主菜单。经过分析, 原因是 raylib 默认将 Esc 键绑定为窗口关闭键, 使得主循环中的 WindowShouldClose() 返回 true。

解决方法是在窗口初始化后禁用默认退出键:

```
1 SetExitKey(0);
```

随后由各个场景自行处理 Esc 键逻辑: 主菜单按 Esc 退出程序, 游戏界面、帮助界面和结算界面按 Esc 返回主菜单。

10.2 音乐与谱面对齐问题

加入音乐后, 如果仍然使用 GetFrameTime() 累加作为游戏时间, 可能会产生音乐和音符不同步的问题。为解决该问题, 程序改为使用音乐播放时间作为统一时钟:

```
1 float chartTime = GetMusicTimePlayed(music) - leadInTime;
```

通过调整 leadInTime, 可以使谱面起点与音乐第一个明显节拍对齐。

10.3 10 Combo 特效重复触发问题

如果直接使用 `combo >= 10` 判断触发特效，会导致特效在 10 Combo 之后每一帧都被重新触发。为解决该问题，程序增加了 `combo10EffectTriggered_` 布尔变量，只允许第一次达到 10 Combo 时触发特效。

10.4 谱面文件异常问题

如果谱面文件不存在或内容非法，程序可能无法正常开始游戏。为提高鲁棒性，项目加入了默认谱面机制：当谱面文件读取失败或没有合法音符时，自动加载内置默认谱面，保证游戏可以继续运行。

11 版本迭代

项目开发过程中使用 Git 进行版本管理，主要版本如下：

表 7: 版本迭代记录

版本	主要内容	说明
v1.0	项目初始化	完成窗口、主菜单和基础项目结构
v2.0	核心玩法增强	完成音符判定、分数统计和 10 Combo 特效
v3.0	音乐同步	加入音乐播放和谱面时间同步机制
v4.0	展示优化	完善结算界面、最高分保存和界面细节

12 AI 工具使用说明

在本项目开发过程中，使用了 AI 工具辅助完成部分设计和调试工作。具体情况如下：

表 8: AI 工具使用说明

工具名称	使用位置	具体用途
ChatGPT	项目方案设计	辅助讨论选题、技术路线、类结构和报告框架
ChatGPT	算法说明	辅助整理音符判定算法、音乐同步方案和谱面格式说明
Codex	代码开发	辅助实现部分功能模块、检查代码问题和小范围修改
ChatGPT	报告撰写	辅助生成实验报告初稿，并由本人根据实际项目进行修改和确认

AI 工具仅作为辅助工具使用。项目的选题确定、功能取舍、代码测试、最终修改和运行验证均由本人完成。

13 收获与总结

通过本次大作业，我对 C++ 图形化程序设计和实时交互程序开发有了更深入的理解。项目不仅涉及基本的类与对象，还使用了继承、多态、STL 容器、文件读写、智能指针和模块化设计等内容。

在游戏逻辑方面，我学习到节奏游戏的关键并不是简单地让音符下落，而是要围绕统一时间轴进行设计。通过使用音乐播放时间作为谱面时间，可以使音符显示、按键判定和音乐播放保持一致。

在工程实践方面，我进一步熟悉了 CMake、raylib 和 Git 的使用，也认识到版本迭代和模块划分的重要性。通过将菜单、游戏、结算等界面拆分为不同场景，程序结构更加清晰，后续扩展也更加方便。

本项目仍有进一步改进空间。例如，可以增加多首歌曲选择、长按音符、谱面编辑器、更加丰富的粒子特效和难度分级系统。整体而言，本项目完成了一个具有完整流程和较好展示效果的 C++ 图形化节奏游戏，达到了本次大作业的预期目标。